

DAY 6 — FINAL CLEAN GENERIC FILTERING LAYER (YOUR TEMPLATE)

✓ GOAL

Move from:

RAW extracted tables ✓

to:

clean retrieval-ready tables ✓

WITHOUT:

- hardcoding ✗
- anchors ✗
- thermal-specific logic ✗
- modifying raw extraction ✗

✓ WHAT DAY 6 DOES

- ✓ removes fully empty rows
- ✓ removes fully empty columns
- ✓ removes useless single-row tables
- ✓ removes useless single-column tables
- ✓ preserves all real data
- ✓ keeps extraction generic
- ✓ improves future LLM retrieval quality

✓ FINAL TEMPLATE STRUCTURE

app/

└─ parser/

| └─ pdfplumber_table.py

| └─ table_segmenter.py

| └─ semantic_propagator.py

| |— table_normalizer.py

| |— quality_scorer.py ✓

|

|— ingestion/

| |— pipeline.py ✓

|

|— tests/

| |— test_ingestion.py ✓

✓ STEP 1 — quality_scorer.py

 **app/parser/quality_scorer.py**

```
import pandas as pd
```

```
def filter_useful_tables(tables):
```

```
    useful = []
```

```
    for t in tables:
```

```
        df = t["df"].copy()
```

```
        # ✓ normalize empty values
```

```
        df = df.map(
```

```
            lambda x: pd.NA
```

```
            if str(x).strip().lower()
```

```
            in ["nan", "none", ""]
```

```
            else x
```

```
        )
```

```
# ✅ remove fully empty columns
```

```
df = df.dropna(axis=1, how="all")
```

```
# ✅ remove fully empty rows
```

```
df = df.dropna(axis=0, how="all")
```

```
rows, cols = df.shape
```

```
# ✅ skip useless tiny tables
```

```
if rows <= 1:
```

```
    continue
```

```
if cols <= 1:
```

```
    continue
```

```
# ✅ update cleaned dataframe
```

```
t["df"] = df
```

```
useful.append(t)
```

```
return useful
```

✅ STEP 2 — UPDATE pipeline.py

 **app/ingestion/pipeline.py**

```
from app.parser.page_builder import (
```

```
    build_page_view
```

```
)
```

```
from app.parser.pdfplumber_table import (  
    extract_tables_pdf  
)
```

```
from app.parser.table_normalizer import (  
    normalize_tables  
)
```

```
from app.parser.quality_scorer import (  
    filter_useful_tables  
)
```

```
def process_pdf(file_path):
```

```
    #  page text
```

```
    pages = build_page_view(file_path)
```

```
    #  raw table extraction
```

```
    raw_tables = extract_tables_pdf(file_path)
```

```
    #  structural normalization
```

```
    normalized_tables = normalize_tables(  
        raw_tables  
    )
```

```
    #  final retrieval-ready filtering
```

```
    useful_tables = filter_useful_tables(
```

```

        normalized_tables

    )

    return {
        "pages": pages,
        "tables": useful_tables
    }

```

✅ STEP 3 — FINAL TEST FILE

 **tests/test_ingestion.py**

```

from app.ingestion.pipeline import process_pdf

```

```

result = process_pdf(
    r"D:\new_train\training\Pdf_praser_llm\data\0183-VP-TDM-06-08196-C006-1339-BRAKE THERMAL
    MAPPING CITY CONDITION Vehicle Dynamics.pdf"
)

```

```

for t in result["tables"]:

```

```

    print("\n=====")

```

```

    print("TABLE:", t["table_id"])

```

```

    print("PAGE:", t["page"])

```

```

    print(t["df"])

```

```

` `

```

✅ WHAT THIS FINAL DAY 6 FIXES

✅ Before

0 1 2 3 4 5 6 7 8

NaN NaN NaN everywhere

tiny useless tables

✔ **After**

- ✔ only meaningful rows
 - ✔ only meaningful columns
 - ✔ all real data preserved
 - ✔ cleaner dataframe
 - ✔ retrieval-ready tables
-

✔ **IMPORTANT DESIGN DECISION**

Extraction Layer

preserve everything ✔

``

Filtering Layer

clean retrieval noise ✔

``

This separation is correct architecture.

✔ **CURRENT PIPELINE (NOW GOOD)**

PDF

↓

PyMuPDF text extraction ✔

↓

pdfplumber table extraction ✔

↓

structural segmentation ✔

↓

semantic propagation 

↓

retrieval filtering 

↓

LLM-ready tables 

 **YOU ARE NOW HERE**

 DAY 6 COMPLETE

Next:

DAY 7

→ embeddings

→ ChromaDB

→ semantic retrieval

→ RAG pipeline